

Flight Pricing Engine

README & Developer Guide

Kamran Sohrab

A dynamic pricing system for general aviation using logistic regression, constrained optimization, and Q-learning.

Current Versions

Project V1.3 Updated April 21, 2026

README V2.2 Updated April 6, 2026

Contents

1	Project Overview	4
2	Features	4
3	System Requirements	5
3.1	Python Version	5
3.2	Required Libraries	5
4	Getting Started: Setup Guide	5
4.1	Step 1 — Verify Python is Installed	5
4.2	Step 2 — Download the Project Files	5
4.3	Step 3 — Create a Virtual Environment (Recommended)	6
4.4	Step 4 — Install Dependencies	6
4.5	Step 5 — Configure Your Aircraft Parameters	6
4.6	Step 6 — Run the Application	7
4.7	Setting Up in an IDE	7
4.7.1	Visual Studio Code	7
4.7.2	PyCharm	8
5	Using the Application	8
5.1	Pricing a Flight — Input Reference	8
5.2	Reading the Output	8
5.3	Recording the Outcome	9
5.4	Viewing the Analytics Report	9
6	Project Structure	9
6.1	Data Flow Summary	10
7	Configuration Reference	10
7.1	Aircraft Limits	10
7.2	Pilot	11
7.3	Operating Costs	11
7.4	Pricing Bounds	11
7.5	Logistic Regression Hyperparameters	11
7.6	MDP / Q-Learning Hyperparameters	12
8	Developer Guide	12
8.1	Architecture Principles	12
8.2	Module-by-Module Reference	13
8.2.1	config.py	13
8.2.2	demand_model.py	13
8.2.3	optimizer.py	13
8.2.4	mdp_agent.py	14
8.2.5	history.py	15
8.2.6	pricer.py	15
8.2.7	utils.py	15
8.3	Adding a New Feature to the Model	15

8.4	Resetting the Models	16
8.5	Backing Up Your Data	16
8.6	Common Errors and Fixes	16
9	Legal Notice	17

Project Overview

The Flight Pricing Engine is a command-line Python application that helps a licensed pilot calculate and dynamically price seat costs for personal flights. Given inputs about a proposed flight — origin, destination, passengers, weights, fuel prices, and duration — the system computes a data-driven ticket price that covers operating costs and optimizes expected revenue based on historical demand patterns.

The system improves over time. Every pricing session that ends in an accept or deny outcome is stored, and two machine learning models update automatically:

- A **logistic regression** model that learns which prices passengers tend to accept on each route and in each context.
- A **Q-learning agent** (Markov Decision Process) that develops a long-run pricing policy by treating each session as a step in a sequential decision problem.

These models are blended with a **constrained nonlinear optimizer** that maximizes expected revenue subject to hard aircraft safety constraints (weight and balance, fuel capacity) on every single quote.

Intended use: This tool is for cost-sharing and pricing *estimation* in a hypothetical or educational context. It does not constitute legal or financial advice. Private pilots are responsible for ensuring compliance with FAR 61.113 (pro-rata cost sharing) and all applicable regulations before accepting any payment from passengers.

Features

- **Weight & Balance check** — automatically verifies that the proposed configuration (fuel + passengers + payload + pilot) does not exceed the aircraft's maximum usable weight before any pricing is computed. No CG (Center of Gravity) calculations are performed as of this version 2.2 of the README or version 1.3 of the software.
- **Fuel LP optimization** — solves a small linear program to determine whether it is cheaper to fuel fully at origin or split fueling across origin and destination, accounting for per-gallon prices at both airports.
- **Dynamic ticket pricing** — computes the revenue-maximizing price per passenger subject to a cost-recovery floor (minimum 5% margin) and a configurable market ceiling.
- **Online learning** — logistic regression coefficients and Q-table values update after every session. The system does not require a batch retraining step.
- **Persistent model state** — learned parameters survive program restarts via two JSON files that are overwritten (not appended) after each session.
- **Analytics report** — a built-in report shows per-route acceptance rates, average prices, total revenue, logistic regression coefficients, and Q-agent policy statistics.
- **No external API required** — all computation is local; no internet connection is needed at runtime.

System Requirements

Python Version

Python **3.8 or higher** is required. The system uses only **f-strings**, **pathlib**, and **typing** features available since Python 3.8.

Required Libraries

Only two third-party libraries are needed beyond the Python standard library:

Library	Version	Purpose
numpy	≥ 1.20	Price grid construction, vectorized probability evaluation
scipy	≥ 1.6	Brent scalar optimizer for price refinement

All other imports (**json**, **math**, **random**, **datetime**, **pathlib**, **typing**) are part of the Python standard library and require no installation.

Getting Started: Setup Guide

This section walks through every step required to go from a fresh machine to a running instance of the pricing engine. Instructions are provided for Windows, macOS, and Linux. Set up instructions for IDEs are also included.

Step 1 — Verify Python is Installed

Open a terminal (Command Prompt or PowerShell on Windows; Terminal on macOS or Linux) and run:

```
python --version
```

You should see output like **Python 3.11.4**. If you see **Python 2.x** or an error, install Python 3 from <https://www.python.org/downloads/>.

Windows users: During the Python installer, check the box that says “*Add Python to PATH*”. Without this, the **python** command will not be recognized in the terminal.

Step 2 — Download the Project Files

Create a folder on your computer where you want the project to live. For example:

```
# macOS / Linux
mkdir ~/projects/flight_pricer
cd ~/projects/flight_pricer
```

```
# Windows (Command Prompt)
mkdir C:\projects\flight_pricer
cd C:\projects\flight_pricer
```

Place all nine `.py` files into this folder. When done, the folder should look exactly like this — no subfolders needed yet (the `data/` directory is created automatically on first run):

```
flight_pricer/  
  config.py  
  demand_model.py  
  history.py  
  main.py  
  mdp_agent.py  
  optimizer.py  
  pricer.py  
  reports.py  
  utils.py
```

Step 3 — Create a Virtual Environment (Recommended)

A virtual environment keeps the project's dependencies isolated from your system Python installation. This is not strictly required, but it is strongly recommended.

```
# macOS / Linux  
python -m venv venv  
source venv/bin/activate  
  
# Windows (Command Prompt)  
python -m venv venv  
venv\Scripts\activate.bat  
  
# Windows (PowerShell)  
python -m venv venv  
venv\Scripts\Activate.ps1
```

You will know the environment is active when your terminal prompt shows `(venv)` at the beginning. To deactivate it later, type `deactivate`.

Step 4 — Install Dependencies

With the virtual environment active, install `numpy` and `scipy`:

```
pip install numpy scipy
```

To confirm the installation succeeded:

```
python -c "import numpy, scipy; print('Dependencies OK')"
```

You should see: `Dependencies OK`.

Step 5 — Configure Your Aircraft Parameters

The software is programmed with parameters for a specific PA-28-236 Piper Dakota aircraft. Before running the system for the first time, open `config.py` in any text editor and update the values in the *Aircraft Limits* and *Pilot* sections to match your actual aircraft and weight.

The most important values to change are:

```

MAX_USABLE_WEIGHT = 1241.23  # lbs -- change to your aircraft's limit
MAX_FUEL_GALLONS   = 72.0     # gal -- change to your usable fuel
      capacity
FUEL_BURN_GPH      = 12.0     # gal/hr -- change to your cruise burn
COST_PER_TACH_HOUR = 115.00   # $    -- change to your actual hourly
      cost
PILOT_WEIGHT_LBS   = 160      # lbs -- change to your actual weight

```

All other settings (pricing bounds, model hyperparameters) can be left at their defaults initially and tuned later. See Section 7 for a full description of every configuration option.

Step 6 — Run the Application

From inside the `flight_pricer/` folder, run:

```
python main.py
```

You will be greeted with a menu:

```

=====
                        FLIGHT PRICING ENGINE  --  MAIN MENU
=====

[1]  Price a new flight
[2]  View analytics & model diagnostics report
[3]  Exit

Select (1/2/3):

```

Select 1 to price your first flight. The `data/` folder and both JSON files will be created automatically at this point.

You're up and running. The system is fully functional from the first session. The machine learning models will warm up and become meaningfully active after approximately 5–10 sessions.

Setting Up in an IDE

Visual Studio Code

1. Open VS Code and select *File* → *Open Folder*, then choose your `flight_pricer/` folder.
2. Install the *Python* extension by Microsoft (search “Python” in the Extensions panel).
3. Press `Ctrl+Shift+P` (or `Cmd+Shift+P` on macOS), type *Python: Select Interpreter*, and choose the interpreter from your virtual environment (`venv`).
4. Open `main.py` and press `F5` to run, or click the *Run* button in the top-right corner.
5. The integrated terminal will appear at the bottom and the application menu will display there.

PyCharm

1. Select *File* → *Open* and choose the `flight_pricer/` folder.
2. Go to *File* → *Settings* → *Project* → *Python Interpreter*. Click the gear icon and select *Add Interpreter* → *Existing Environment*, then point it to the Python executable inside your `venv` folder.
3. Right-click `main.py` in the project tree and select *Run* 'main'.
4. PyCharm will run the script in its integrated terminal at the bottom of the screen.

Important for all IDEs: The application is interactive and reads input from `stdin`. Make sure your IDE is running the script in an *interactive terminal*, not just a plain output console. In VS Code, set "`terminal.integrated.defaultProfile`" if needed. In PyCharm, ensure *Emulate terminal in output console* is checked under *Run* → *Edit Configurations*.

Using the Application

Pricing a Flight — Input Reference

When you select option 1 from the main menu, the system will prompt for the following inputs in order:

Table 1: Input prompts and expected values.

Prompt	Format	Notes
Origin airport	Text	ICAO (e.g. KCMJ), IATA, or plain name
Destination airport	Text	Same formats as origin
Departure date & time	YYYY-MM-DD HH:MM	Used to compute booking lead time
Round trip?	y / n	Affects fuel LP and tach-hour calculation
Number of passengers	Integer	Excludes pilot
Average passenger weight	Decimal (lbs)	Used for weight & balance check
Total payload weight	Decimal (lbs)	Baggage and cargo; maximum 200 lbs
Fuel price at origin	Decimal (\$/gal)	Used in fuel LP objective
Fuel price at destination	Decimal (\$/gal)	Used in fuel LP objective
Flight duration (one way)	Decimal (hours)	Tach-hour cost and fuel burn are derived from this

Reading the Output

After computing, the system displays five result sections:

1. **Weight & Balance** — total ramp weight with a component breakdown (fuel, passengers, payload, pilot) and the margin below the max usable weight limit. If overweight, the session

stops here.

2. **Fuel Recommendation (LP Solution)** — the optimal fueling strategy (origin-only, origin-full, or split), gallons at each stop, and total fuel cost.
3. **Operating Costs** — tach hours, aircraft cost, fuel cost, total operating cost, and cost per seat.
4. **Model Status** — how many SGD updates the logistic regression has processed, how many Q-learning episodes the agent has experienced, the current exploration rate ε , and the recent route acceptance rate.
5. **Ticket Pricing** — the NLP-optimal price, the blended final price, estimated acceptance probability $P(\text{accept})$, expected revenue, and expected profit.

Recording the Outcome

After reviewing the price, you will be asked:

```
Was this pricing accepted by the passenger(s)? (y/n):
```

Enter **y** if the passengers agreed to the quoted price, or **n** if they declined. **This step is critical** — it is the feedback that trains both machine learning models. Never skip it, even for hypothetical sessions.

Viewing the Analytics Report

Select option 2 from the main menu to see:

- Overall totals: quotes, accepted, denied, total profit.
- Per-route breakdown: acceptance rate, average price, average revenue, total revenue.
- Current logistic regression coefficients with direction indicators.
- Q-agent episode count, current ε , and a sample of non-zero Q-values from the policy table.

Project Structure

```
flight_pricer/
|
+-- main.py           Entry point. Run this file.
+-- config.py         All constants and hyperparameters.
+-- demand_model.py   Layer 1: logistic regression.
+-- optimizer.py      Layer 2: fuel LP + NLP revenue optimizer.
+-- mdp_agent.py      Layer 3: Q-learning MDP agent.
+-- pricer.py         Orchestrates all three layers per session.
+-- history.py        JSON read/write and route statistics.
+-- reports.py        Analytics and model diagnostics report.
+-- utils.py          CLI prompts, colors, and formatting.
|
+-- data/             (auto-created on first run)
    +-- flight_history.json  Append-only log of all sessions.
    +-- model_state.json    Overwritten snapshot of model weights.
```

Data Flow Summary

1. `main.py` receives user menu selection and delegates.
2. `pricer.py` collects inputs and orchestrates the session.
3. `optimizer.py` solves the fuel LP and weight check, then runs the NLP price optimizer using probabilities from `demand_model.py`.
4. `mdp_agent.py` provides a Q-learning price suggestion that is blended with the NLP result.
5. `history.py` retrieves the route acceptance rate (used in state construction) and persists the completed record.
6. After outcome entry, `demand_model.py` and `mdp_agent.py` both update their internal parameters and overwrite `model_state.json`.

Configuration Reference

All tunable parameters live in `config.py`. The file is organized into six groups. See the following tables for a description of each tunable parameter as well as the default values selected (you may adjust all values to fit you best)

Aircraft Limits

Constant	Default	Description
SEATS_AVAILABLE	3	Paying passenger seats (total minus pilot)
MAX_USABLE_WEIGHT	1241.23	Combined weight limit in lbs
MAX_FUEL_GALLONS	72.0	Usable fuel capacity in gallons
FUEL_LBS_PER_GALLON	6.0	Avgas weight (lbs per gallon)
FUEL_BURN_GPH	12.0	Cruise fuel burn (gal/hr)
MAX_PAYLOAD_LBS	200.0	Hard baggage/cargo limit (lbs)
RESERVE_HOURS	0.75	VFR/IFR fuel reserve requirement (hours)

Pilot

Constant	Default	Description
PILOT_WEIGHT_LBS	160	Pilot weight included in W&B calculation

Operating Costs

Constant	Default	Description
COST_PER_TACH_HOUR	115.00	Aircraft cost per tach hour (\$)
TACH_TO_FLIGHT_HOUR_RATIO	0.80	Long-run tach-to-flight-hour conversion

Pricing Bounds

Constant	Default	Description
MIN_MARGIN	0.05	Minimum margin above per-seat cost (5%)
PRICE_CEILING_MULTIPLIER	5.0	Max price = 5× per-seat cost
PRICE_GRID_STEPS	200	Price grid resolution for NLP sweep

Logistic Regression Hyperparameters

Constant	Default	Description
LR_LEARNING_RATE	0.05	SGD step size α
LR_REGULARIZATION	0.01	L2 penalty coefficient λ
LR_MIN_SAMPLES	5	Records before model activates
LR_INITIAL_BETAS	[2.0, -0.01, -0.1, 0.005, -0.05, 0.1]	Initial coefficient vector

MDP / Q-Learning Hyperparameters

Constant	Default	Description
MDP_DISCOUNT_FACTOR	0.92	Future reward discount γ
MDP_LEARNING_RATE	0.15	Bellman update step size α_Q
MDP_EPSILON_START	0.25	Initial exploration rate ϵ_0
MDP_EPSILON_MIN	0.02	Exploration rate floor
MDP_EPSILON_DECAY	0.97	Per-episode decay multiplier
MDP_PRICE_BINS	12	Discrete price multiplier actions
MDP_STATE_BINS	[4,4,3,4]	State space bin counts

Developer Guide

This section is intended for contributors or anyone who wants to modify, extend, or debug the codebase.

AI/LLM Usage Policy: Please do not use Large Language Models or Artificial Intelligent Chatbots. AI-Assistants like Claude Code and Github Copilot may be utilized but are strongly discouraged. This is a college student's passion project, and while AI and LLMs may implement features with ease, they can also hallucinate and damage the logic behind this model. Additionally, non-original code goes against the spirit of this *personal passion* project, please read my Technical Report to see my construction of this model (It took a lot of work and critical thinking).

Architecture Principles

The codebase follows three design rules that should be preserved in any extension:

1. **Maintain strict layer separation.** `demand_model.py`, `optimizer.py`, and `mdp_agent.py` do not import from each other. They communicate only through `pricer.py`, which acts as the orchestrator. If you add a new model layer, add it as a new module and wire it through `pricer.py`.
2. **Config.py is the single source of truth.** No reused numbers appear in any module other than `config.py`. If you find yourself hardcoding a threshold or weight value anywhere else, move it to `config.py` first.
3. **Persistence is explicit.** Both `demand_model.py` and `mdp_agent.py` call their own `_save_state()` methods immediately after updating parameters. Do not save state from `pricer.py` or any other module.

Module-by-Module Reference

`config.py`

Contains only constants. No functions, no classes, no logic. Import individual constants using explicit names:

```
from config import MAX_USABLE_WEIGHT, FUEL_BURN_GPH
```

Never import `config` as a module object; this makes dependency tracking harder.

`demand_model.py`

Defines the `LogisticDemandModel` class. Key methods:

Method	Description
<code>predict.accept_prob(.)</code>	Returns $P(\text{accept}) \in [0, 1]$ for a single price/feature set. Returns 0.5 if fewer than <code>LR_MIN_SAMPLES</code> updates have occurred.
<code>predict.accept_prob_curve(grid, .)</code>	Vectorized version over a numpy price array. Used by the optimizer.
<code>update(., accepted: bool)</code>	Performs one SGD step and saves state. Call this after every session.
<code>summary()</code>	Returns a dict with current betas, update count, and human-readable interpretation. Used by <code>reports.py</code> .

`optimizer.py`

Contains only pure functions (no classes, no state). Key functions:

Function	Description
<code>solve_fuel_lp(.)</code>	Evaluates the two closed-form LP strategies and returns the cheaper one as a dict with keys <code>strategy</code> , <code>gallons_origin</code> , <code>gallons_dest</code> , <code>total_fuel_cost</code> , <code>note</code> .
<code>check_weight_constraint(.)</code>	Returns a dict with <code>feasible</code> (bool), <code>total_lbs</code> , <code>slack_lbs</code> , and <code>breakdown</code> .
<code>max_gallons_within_weight(.)</code>	Computes G_{wb} — the maximum gallons loadable without violating the weight limit. Pass this to <code>solve_fuel_lp</code> .
<code>compute_operating_cost(.)</code>	Returns tach hours, aircraft cost, fuel cost, and total operating cost as a dict.
<code>optimize_price(.)</code>	Main pricing function. Accepts a <code>LogisticDemandModel</code> instance and returns the full pricing result dict including <code>final_price_per_pax</code> , <code>expected_revenue</code> , and the raw price/revenue curve arrays.

`mdp_agent.py`

Defines the `QLearningAgent` class and several standalone discretization functions. Key components:

Component	Description
<code>build_state(.)</code>	Takes acceptance rate, lead days, num passengers, and departure month; returns a <code>(int, int, int, int)</code> state tuple.
<code>action_to_multiplier(k)</code>	Maps action index $k \in [0, N_A - 1]$ to a price multiplier in $[0.60, 1.40]$.
<code>suggest_price_multiplier(state)</code>	Returns the greedy multiplier and mode string ("exploit") with no exploration. Use this for the final price suggestion in <code>pricer.py</code> .
<code>select_action(state)</code>	Returns an ε -greedy action for training.
<code>record_outcome(s, a, r, s')</code>	Applies the Bellman update and saves state.
<code>policy_summary()</code>	Returns the full greedy policy table as a dict. Used by <code>reports.py</code> .

history.py

Defines the FlightHistory class. Key methods:

Method	Description
<code>append(record: dict)</code>	Appends a session record to <code>flight_history.json</code> .
<code>all_records()</code>	Returns the full list of session records.
<code>records_for_route(key)</code>	Filters records to a specific route key (e.g. KCMI-KORD).
<code>route_acceptance_rate(key)</code>	Returns the acceptance rate for the most recent 20 records on a route. Returns 0.5 if fewer than 2 records exist.
<code>compute_lead_days(date_str)</code>	Parses a date string and returns days until departure.
<code>route_summary()</code>	Returns per-route aggregate statistics for <code>reports.py</code> .

pricer.py

The `run_pricing_session()` function is the only public interface. It instantiates all three model classes, runs the 14-step session flow, and returns nothing (all side effects are persistence). If you want to add a new input field, this is the only file you need to touch in addition to `config.py` and the relevant model file.

utils.py

Provides `prompt_float()`, `prompt_int()`, `prompt_str()`, `prompt_bool()`, `format_currency()`, `print_header()`, `print_section()`, `print_warning()`, `print_ok()`, and `print_info()`. These are pure I/O helpers with no business logic. Edit this file only to change the CLI appearance.

Adding a New Feature to the Model

To add a new input feature to the logistic regression (for example, day of week), follow these steps:

1. In `config.py`, add a new initial beta value to `LR_INITIAL_BETAS` (e.g. 0.02).
2. In `demand_model.py`, update `_build_feature_vector()` to include the new feature as an additional element in the returned list. Update the `feature_names` list in `summary()` to match.
3. In `pricer.py`, add a prompt to collect the new input and pass it through to `lr_model.update()` and `demand_model.predict_accept_prob()`.
4. **Delete** `data/model_state.json` before the next run to reset the model with the new coefficient vector. The old betas have the wrong dimensionality and will cause errors.

Breaking change: Adding or removing features changes the length of the beta vector. Any existing `model_state.json` trained on the old feature set must be deleted before running with the updated code. `flight_history.json` is not affected and does not need to be deleted.

Resetting the Models

To reset one or both learned models without losing flight history:

- **Reset both models:** Delete `data/model_state.json`. The file will be recreated from `LR_INITIAL_BETAS` and a zeroed Q-table on the next run.
- **Reset only the logistic regression:** Open `model_state.json` and delete the "logistic_regression" key, then save.
- **Reset only the Q-agent:** Open `model_state.json` and delete the "q_learning" key, then save.
- **Reset everything including history:** Delete the entire `data/` folder. Both files will be recreated on next run.

Backing Up Your Data

The `data/` folder contains everything the system has learned. Back it up regularly, especially `flight_history.json`. A simple approach is to copy the folder with a date suffix:

```
# macOS / Linux
cp -r data/ data_backup_2025-07-04/

# Windows
xcopy data data_backup_2025-07-04 /E /I
```

Common Errors and Fixes

Error	Fix
<code>ModuleNotFoundError: numpy</code>	Run <code>pip install numpy scipy</code> with your virtual environment active.
<code>ModuleNotFoundError: config</code>	Run the script from inside the <code>flight_pricer/</code> folder, not from a parent directory.
<code>JSONDecodeError</code> on startup	One of the JSON files in <code>data/</code> is corrupt. Delete <code>model_state.json</code> (and <code>flight_history.json</code> if needed) and restart.
<code>ValueError</code> from numpy in optimizer	Usually caused by a mismatched beta vector length. Delete <code>model_state.json</code> and restart.
Input not being read (IDE)	Enable interactive terminal mode in your IDE. See Section 4.2.

Legal Notice

This software is provided for **educational and hypothetical use only**. It does not constitute legal, financial, or aeronautical advice. The author makes no warranty that prices generated by this system comply with any applicable regulation. All pilots must independently check the outputs of this software and complete all preflight responsibilities. All pilots must independently ensure their proposed flight is safe, legal, and must exercise proper aeronautical decision-making. Refer to your aircraft's Pilot Operating Handbook for performance charts, details, and a complete list of preflight actions.

Pilots operating under FAR Part 61 are solely responsible for ensuring compliance with **FAR 61.113** regarding cost sharing with passengers. Accepting compensation beyond a pro-rata share of operating costs may constitute acting as a commercial operator and require a commercial pilot license and potentially an Air Carrier Certificate under FAR Part 119.

Always consult the FAR, a certificated flight instructor, aviation attorney, and/or the FAA before accepting any payment from passengers.